INTELIGÊNCIA ARTIFICIAL

Algoritmos de Machine Learning

Gabriel de Souza Dourado · 12 de abril de 2026

1 K-Nearest Neighbors (KNN)

O KNN é um algoritmo de aprendizado supervisionado usado tanto para problemas de classificação quanto de regressão. É classificado como *Lazy Learning* pois, diferentemente dos algoritmos *eager*, que constroem um modelo explícito durante o treinamento, o KNN adia todo o processamento para a fase de predição, limitando-se a memorizar o conjunto $\mathcal{D}$.

Algorithm 1 KNN-Treino

Entrada: Conjunto de dados $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$
Saída: Dados armazenados na memória

```
return D
```

Algorithm 2 KNN-Predição (Classificação)

Entrada: Amostra $\mathbf{z} \in \mathbb{R}^D$, inteiro $K \geq 1$, conjunto $\mathcal{D}$ armazenado na memória
Saída: Classe predita $\hat{y}$

```
for i = 1 até N do
    d_i ← ||z − x_i||_2
end for
N_K(z) ← índices dos K menores valores em {d_i}
ŷ ← arg max_c Σ_{i∈N_K(z)} 1[y_i = c]
return ŷ
```

Algorithm 3 KNN-Predição (Regressão)

Entrada: Amostra $\mathbf{z} \in \mathbb{R}^D$, inteiro $K \geq 1$, conjunto $\mathcal{D}$ armazenado na memória
Saída: Classe predita $\hat{y}$

```
for i = 1 até N do
    d_i ← ||z − x_i||_2
end for
N_K(z) ← índices dos K menores valores em {d_i}
ŷ ← (1/k) Σ_{i∈N_K(z)} x_i
return ŷ
```

2 Naive Bayes Classifier

O Classificador Naive Bayes é um algoritmo de aprendizado supervisionado baseado no Teorema de Bayes com a suposição de independência condicional entre os atributos dado a classe. Formalmente, dado um vetor de atributos $\mathbf{x} = (x_1, x_2, \ldots, x_D)$ e um conjunto de classes $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$, o modelo estima:

$$P(y = c_k \mid \mathbf{x}) \;\propto\; P(y = c_k) \prod_{d=1}^{D} P(x_d \mid y = c_k)$$

A predição é dada pela regra MAP (Maximum A Posteriori):

$$\hat{y} = \arg\max_{c_k \in \mathcal{C}} \left[\log P(y = c_k) + \sum_{d=1}^{D} \log P(x_d \mid y = c_k) \right]$$

Algorithm 4 NaiveBayes-Treino

Entrada: Conjunto de dados $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$, conjunto de classes $\mathcal{C} = \{c_1, \ldots, c_K\}$
Saída: Probabilidades a priori $\hat{\pi}_k$ e verossimilhanças $\hat{\theta}_{kd}$

```
for cada classe c_k ∈ C do
    D_k ← {(x_i, y_i) ∈ D | y_i = c_k}
    π̂_k ← |D_k| / N
    for d = 1 até D do
        Estimar θ̂_kd ← P(x_d | y = c_k) a partir de D_k
    end for
end for
return {π̂_k}, {θ̂_kd}
```

Algorithm 5 NaiveBayes-Predição

Entrada: Amostra $\mathbf{z} = (z_1, \ldots, z_D) \in \mathbb{R}^D$, parâmetros $\{\hat{\pi}_k\}$ e $\{\hat{\theta}_{kd}\}$
Saída: Classe predita $\hat{y}$

```
for cada classe c_k ∈ C do
    s_k ← log π̂_k
    for d = 1 até D do
        s_k ← s_k + log P(θ̂_kd)
    end for
end for
ŷ ← arg max_{c_k} s_k
return ŷ
```

3 Decision Trees

Consiste em um algoritmo de aprendizado supervisionado usado tanto para problemas de classificação quanto de regressão. Formalmente, uma Árvore de Decisão é um **Grafo Acíclico Direcionado (DAG)** onde cada **nó interno** representa uma condição sobre um atributo, cada **aresta** representa o resultado dessacondição e cada **folha** armazena a predição final.

Algorithm 6 ClassTree-MelhorDivisão

Entrada: Subconjunto $\mathcal{D}_m = \{(\mathbf{x}_i, y_i)\}$ no nó m, critério $Q \in \{\text{Gini, Entropia}\}$
Saída: Atributo ótimo j^*, ponto de corte ótimo t^* e impureza mínima ponderada

menor_imp $\leftarrow \infty$; $j^* \leftarrow$ nulo; $t^* \leftarrow$ nulo
for $j = 1$ **até** D **do**
    Ordenar $\{x_{ij}\}$ e gerar candidatos t (médias entre valores consecutivos)
    for cada candidato t **do**
        $R_L \leftarrow \{\mathbf{x} \in \mathcal{D}_m \mid x_j \leq t\}$; $R_R \leftarrow \{\mathbf{x} \in \mathcal{D}_m \mid x_j > t\}$
        imp $\leftarrow \dfrac{|R_L|}{|\mathcal{D}_m|} Q(R_L) + \dfrac{|R_R|}{|\mathcal{D}_m|} Q(R_R)$
        if imp $<$ menor_imp **then**
            menor_imp $\leftarrow$ imp; $j^* \leftarrow j$; $t^* \leftarrow t$
        end if
    end for
end for
return (j^*, t^*, menor_imp)

Algorithm 7 ClassTree-Construção (Recursivo)

Entrada: Subconjunto $\mathcal{D}_m$ e critério Q,
Saída: Nó (folha ou interno) da árvore

$\hat{p}_{mk} \leftarrow \dfrac{|\{i : y_i = k\}|}{|\mathcal{D}_m|}$ para todo k
if critério de parada($\mathcal{D}_m$) for verdadeiro **then**
    return Folha$\left(\hat{y}_m \leftarrow \arg\max_k \hat{p}_{mk}\right)$
end if
(j^*, t^*, imp_filhos) $\leftarrow$ ClassTree-MelhorDivisão($\mathcal{D}_m$, Q)
$\Delta \leftarrow Q(\mathcal{D}_m) -$ imp_filhos
if $\Delta < \epsilon$ **then**
    return Folha$\left(\hat{y}_m \leftarrow \arg\max_k \hat{p}_{mk}\right)$
end if
$R_L \leftarrow \{\mathbf{x} \in \mathcal{D}_m \mid x_{j^*} \leq t^*\}$; $R_R \leftarrow \{\mathbf{x} \in \mathcal{D}_m \mid x_{j^*} > t^*\}$
filho_esq $\leftarrow$ ClassTree-Construção(R_L, Q)
filho_dir $\leftarrow$ ClassTree-Construção(R_R, Q)
return NóInterno(j^*, t^*, filho_esq, filho_dir)

Algorithm 8 ClassTree-Predição

Entrada: Amostra $\mathbf{z} \in \mathbb{R}^D$, árvore T
Saída: Classe predita $\hat{y}$

```
nó ← raiz de T
while nó não é folha do
    (j*, t*) ← atributo e corte armazenados em nó
    if z_{j*} ≤ t* then
        nó ← nó.filho_esq
    else
        nó ← nó.filho_dir
    end if
end while
return ŷ_m armazenado na folha
```

Referências

[1] FACELI, K.; LORENA, A. C.; GAMA, J.; CARVALHO, A. C. P. L. F. *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina*. Rio de Janeiro: LTC, 2011.

[2] HASTIE, T.; TIBSHIRANI, R.; FRIEDMAN, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2. ed. New York: Springer, 2009.

[3] MITCHELL, T. M. *Machine Learning*. New York: McGraw-Hill, 1997.

[4] GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge: MIT Press, 2016.

[5] BISHOP, C. M. *Pattern Recognition and Machine Learning*. New York: Springer, 2006.